

Optimisation par Colonie de Fourmis d'un Chemin sur un Paysage Fractal

Rapport de Parallélisation (OpenMP & MPI)

Alfredo QUINTELLA PINTO Felipe BIASUZ CORDEIRO
Naomy Isabela GOMES BIAGGI

13 mars 2026

1 Introduction

Ce rapport présente l'optimisation et la parallélisation d'un algorithme de colonie de fourmis (Ant Colony Optimization, ACO) appliqué à la recherche d'un chemin optimal sur un terrain fractal. L'objectif principal est de réduire le temps de calcul de la simulation en exploitant différentes formes de parallélisme.

Deux approches de calcul parallèle ont été étudiées :

- la parallélisation en mémoire partagée avec OpenMP,
- la parallélisation en mémoire distribuée avec MPI.

Le travail a consisté à identifier les sections critiques du code, vectoriser les structures de données puis paralléliser les parties les plus coûteuses en temps de calcul.

2 Modèle de Colonies de Fourmis

Le modèle utilisé simule un ensemble de fourmis artificielles évoluant sur une grille 2D représentant l'environnement. Chaque cellule de la grille possède une valeur indiquant le coût temporel de traversée du terrain.

Deux types de phéromones sont utilisés :

- $V_1(s)$: phéromone d'exploration indiquant les chemins menant vers la nourriture.
- $V_2(s)$: phéromone indiquant le chemin de retour vers la fourmilière.

Chaque fourmi peut être dans deux états :

- non chargée (recherche de nourriture),
- chargée (retour vers la fourmilière).

À chaque itération, une fourmi :

1. met à jour les valeurs de phéromones de la cellule courante en fonction des cellules voisines,
2. choisit une cellule voisine vers laquelle se déplacer,
3. met à jour son état si elle atteint la nourriture ou la fourmilière.

Le déplacement est déterminé de manière probabiliste :

- avec une probabilité ε , la fourmi explore une direction aléatoire,

- avec une probabilité $1 - \varepsilon$, elle suit la cellule ayant la concentration de phéromone la plus élevée.
- Les phéromones subissent également un phénomène d'évaporation à chaque itération.

3 Vectorisation du Modèle

Afin de préparer le code à la parallélisation, les structures de données représentant les fourmis ont été vectorisées.

Initialement, chaque fourmi pouvait être représentée par une structure contenant ses propriétés. Dans la version optimisée, les données ont été transformées en une structure de tableaux (Structure of Arrays).

Les propriétés suivantes sont stockées dans des vecteurs :

- position x des fourmis,
- position y des fourmis,
- état de chaque fourmi,
- graine du générateur aléatoire.

Cette organisation permet :

- une meilleure localité mémoire,
- une vectorisation plus efficace par le compilateur,
- une parallélisation plus simple des boucles sur les fourmis.

4 Structure du Code

L'architecture du programme repose sur plusieurs classes principales :

- **ants** : gestion des positions, états et déplacements des fourmis.
- **pheromone** : stockage et mise à jour des phéromones sur la grille.
- **fractal_land** : génération du terrain fractal et gestion des coûts de déplacement.

La fonction principale responsable de l'évolution de la simulation est `advance_time`, qui appelle la fonction `ant::advance` pour chaque fourmi.

5 Profilage de la Version Séquentielle

Une analyse de performance a été réalisée à l'aide de l'outil `gprof`.

Le profil plat indique que la majorité du temps d'exécution est concentrée dans la fonction responsable du déplacement des fourmis :

- `ant::advance` : **61.48 %** du temps total
- `Renderer::display` : 26.70 %
- `advance_time` : 11.24 %

Le temps moyen par itération pour le déplacement séquentiel était de 2.48 ms.

Ces résultats montrent que la parallélisation doit se concentrer en priorité sur la boucle de déplacement des fourmis.

6 Parallélisation en Mémoire Partagée (OpenMP)

La boucle principale traitant les fourmis a été parallélisée avec la directive :

```
#pragma omp parallel for schedule(guided)
```

Le choix de la politique `guided` permet une meilleure répartition dynamique de la charge de travail entre les threads. En effet, certaines fourmis peuvent effectuer plus d'opérations que d'autres selon leur position et l'état du terrain.

L'utilisation de directives `atomic` a été nécessaire pour éviter les conditions de course (data race) lors de la mise à jour des phéromones, notamment au début de la simulation lorsque toutes les fourmis se trouvent dans le nid.

6.1 Résultats de Performance

Le tableau 1 présente le temps de déplacement total des fourmis (pour 2585 itérations) en fonction du nombre de threads. Le Speedup (S) est calculé par rapport à l'exécution avec 1 thread (5781.72 ms).

Threads	Temps déplacement (ms)	Speedup	Temps évaporation (ms)
1	5781.72	1.00	1200.49
2	4027.83	1.43	1255.97
4	2595.99	2.22	1534.09
6	1981.15	2.91	1562.31
10	1573.26	3.67	1792.22
14	1710.82	3.37	1798.17
16	5874.92	0.98	1691.49

TABLE 1 – Performances OpenMP

6.2 Analyse avec la loi d'Amdahl

La loi d'Amdahl permet d'expliquer la saturation du speedup :

$$S(N) = \frac{1}{(1 - p) + \frac{p}{N}}$$

où p représente la fraction parallélisable du programme.

Dans notre cas, certaines parties du code restent séquentielles (gestion des phéromones, synchronisation), ce qui limite le gain maximal.

La forte dégradation observée avec 16 threads s'explique principalement par :

- la contention mémoire sur les variables partagées,
- le coût des opérations `atomic`,
- la saturation de la bande passante mémoire.

7 Parallélisation en Mémoire Distribuée (MPI)

Pour la parallélisation MPI, la première stratégie proposée a été implémentée : chaque processus conserve la carte entière mais ne gère qu'une fraction des fourmis. À la fin de chaque itération, une réduction globale (`MPI_Allreduce` avec l'opérateur `MPI_MAX`) est effectuée pour synchroniser la carte des phéromones entre tous les processus.

7.1 Résultats

Processus	Temps déplacement (ms)	Speedup	Temps communication (ms)
1	5781.72	1.00	-
2	3287.85	1.76	3791.86
3	1963.10	2.94	3890.58
4	2701.99	2.14	5479.22
6	7219.13	0.80	8489.75
8	17792.30	0.32	14872.80

TABLE 2 – Performances MPI

7.2 Analyse MPI

Le modèle MPI offre d'excellentes performances pour le calcul strict du déplacement (Compute Bound) jusqu'à 3 processus, abaissant le temps à 1963 ms.

Néanmoins, l'architecture révèle rapidement son point faible : **le coût des communications**. À partir de 4 processus, le temps d'exécution explose. Avec 8 processus, l'évaporation et la synchronisation prennent près de 15 secondes. Cela s'explique par la taille de la carte des phéromones. Lors de l'appel à `MPI_Allreduce`, une quantité massive de données (plusieurs mégaoctets par itération) doit transiter par le réseau ou la mémoire partagée du système, transformant l'application en un problème fortement limité par les communications (Network Bound).

8 Conclusion

Ce projet illustre les compromis classiques du calcul haute performance entre parallélisme de calcul et coût de communication.

La parallélisation OpenMP s'est révélée la plus efficace pour ce problème, avec un speedup maximal de **3.67x**. L'approche MPI permet également d'accélérer le calcul lorsque le nombre de processus reste faible, mais elle devient rapidement limitée par le coût des communications globales.

Une amélioration future consisterait à utiliser une approche hybride MPI + OpenMP, permettant de limiter le nombre de processus MPI tout en exploitant efficacement les cœurs de calcul disponibles via le multithreading.